Hertie School
Algorithmic Game Theory and Governance
GRAD-E1489

Stochastic Multi-Armed Bandits: Theory and Implementation

Term Paper

`nicolasreichardt/stochastic-bandit-algorithms`

Name: Nicolas Reichardt
Student ID: 245611
Instructor: Prof. Asya Magazinnik, PhD

Submission Date: January 5, 2026

1. Introduction

This term paper provides an introduction to Stochastic multi-armed bandits (MAB) as a framework for decision-making under uncertainty. Multi-armed bandits model situations in which an agent repeatedly chooses among a set of actions (or "arms") where each arm is associated with an unknown reward distribution and the overarching goal is to maximize the cumulative reward over time. From a game-theoretic perspective, multi-armed bandits can be interpreted as a repeated game, where an agent must learn optimal strategies over time without having full information.

There exist different types of MAB settings, such as stochastic, adversarial and contextual bandits. In this paper, we will focus on the stochastic setting, in which each arm is assumed to generate rewards independently from a fixed but unknown probability distribution. The goal of this paper is to provide a high-level overview of the main MAB algorithms and their implementations.

For that, we will first provide a clear overview of the multi-armed bandit problem, its origins, its purpose and typical areas of application. Subsequently, we will present the theoretical and probabilistic foundations required to understand three widely used algorithms: (1) ε-greedy, (2) Upper Confidence Bound (UCB) and (3) Thompson sampling. We will then explain the intuition behind these algorithms, as well as how they are implemented and how they operate in practice. In the final part, we will implement the algorithms from scratch, we will evaluate their performance on a toy example and compare the results and discuss the observed differences.

2. Multi-Armed Bandits and their Application

The term multi-armed bandits is inspired by the analogy of a gambler facing several slot machines ("one-armed bandits"). This analogy captures the central challenge faced by the decision-maker: each action produces uncertain outcomes and information can only be acquired by actively making choices between the different available options. In general, the multi-armed bandit problem is concerned with finding a balance between "exploration" and "exploitation". This means that on the one hand, MAB are trying out different options to learn about their rewards and, on the other hand, they are exploiting the currently best-known option in order to maximize the total reward. Because this exploration–exploitation trade-off is present in many real-world decision-making problems, MAB models have a wide range of practical applications.

One typical example is optimal resource allocation, where a limited budget or resources have to be distributed among several alternatives whose performance is not known in advance and must be learned over time (Botosan and Bilokon, 2024). For instance, it can be used in education research and education policy where different interventions (such

as tutoring programs, incentive schemes or teaching methods) are tested and adaptively assigned to schools or students based on observed outcomes. This allows policymakers to learn which interventions are most effective while reducing the cost of experimenting with less effective options (Clement et al., 2015; Manickam et al., 2017; Rafferty et al., 2019)

A second widely used application is in online learning and sequential decision-making problems. These include, for instance, recommendation systems, where platforms must decide which items (such as movies, products or news articles) to recommend to users in order to maximize engagement or user satisfaction. MAB algorithms are particularly well suited for these problems because they allow the system to continuously learn from user feedback and adapt recommendations over time (Li et al., 2010; Mehrotra et al., 2020)

Furthermore, multi-armed bandits are very common in the online advertising industry or as an alternative to traditional A/B testing. In digital advertising, different ads, website layouts or pricing strategies can be modeled as arms and user interactions such as clicks or conversions as rewards. In contrast to standard A/B tests, which allocate traffic uniformly for a fixed period of time, bandit algorithms can adaptively assign more users to better-performing options while still exploring other alternatives. This leads to faster learning and lower opportunity costs (Gigli and Stella, 2025; Schwartz et al., 2017).

As already mentioned, different types of MAB settings can be distinguished. Stochastic bandits assume fixed but unknown reward distributions. Adversarial bandits consider settings with potentially non- or worst-case rewards (rewards can be chosen by an adversary, potentially changing over time). Contextual bandits extend the model by including additional information that can be used to guide decisions.

In the stochastic setting, a number of algorithms have been developed over the last several decades. Their main difference lies in how they handle uncertainty and how they encourage the exploration. Before discussing Upper Confidence Bound (UCB) and Thompson Sampling in detail, we will also introduce and implement the ε-greedy algorithm, which serves as a simple and intuitive baseline.

3. Theoretical and Probabilistic Foundations of Stochastic Bandits

In this paper, we will focus on stochastic bandits, meaning that rewards are i.i.d. and each arm has a fixed but unknown reward distribution. The goal of stochastic bandit algorithms is to balance the gain of new information, which could improve future performance and the gain from choosing the currently best option, which is already known to perform well. As an example, one could take a current data science student, who is trying to balance between figuring out which skills to learn next to improve their job market prospects (exploration) and improving a current skill that is already known to be

in demand (exploitation).

But how to best balance between exploration and exploitation? And how to maximise the cumulative reward? This is what MAB algorithms are trying to solve and to answer this, we need to formulate this problem setting in a mathematically sound way. The following section provides the mathematical foundations to understand MAB algorithms and is based on Ashwin Rao's lecture on multi-armed bandits at Stanford University (2021).

To achieve a high-level understanding of the different algorithms, let us consider a set of m actions (or *arms*) $\mathcal{A} = \{1, 2, \ldots, m\}$. Each arm a gives a reward r_t drawn from an unknown distribution R_a, usually bounded between 0 and 1:

$$r_t \sim R_{a_t}.$$

At each time step t the agent:

1. Picks an arm $a_t \in \mathcal{A}$.
2. Observes a reward $r_t \sim R_{a_t}$.
3. Updates its strategy based on past actions and rewards.

The agent's history up to time t is $h_t = (a_1, r_1, \ldots, a_t, r_t)$. A policy uses this history to decide which arm to choose next. A policy is a rule that tells the agent how to choose the next arm based on the actions and rewards it has seen so far.The goal is to maximize total reward:

$$E\Big[\sum_{t=1}^{T} r_t\Big]$$

or equivalently to minimize cumulative regret, which measures how much reward is lost by not always picking the best arm.

The expected reward of an arm is its *action value*:

$$Q(a) := E[r \mid a].$$

The action value is the expected reward for picking a specific arm and is the main quantity that bandit algorithms try to estimate and compare. The best arm a^* has the highest $Q(a)$ and the gap of any arm a is $\Delta_a = Q(a^*) - Q(a)$.

Cumulative regret can be expressed as:

$$L_T = \sum_{a \in \mathcal{A}} E[N_T(a)]\, \Delta_a$$

$N_T(a)$ counts how often arm a is chosen. This shows the challenge: we want to pick suboptimal arms as little as possible while learning which arm is best. In addition, most algorithms track the empirical mean rewards $\hat{Q}_t(a)$ to estimate the value of each arm and balance exploration and exploitation.

Common approaches to maximize the total reward include the "ϵ-Greedy" algorithm, which mostly exploits the best-known arm but sometimes explores randomly, "UCB1", which picks arms based on both estimated value and uncertainty and "Thompson Sampling", which samples from a probability distribution over each arm's potential reward. All of these methods handle the exploration–exploitation tradeoff in different ways.

Now that we have a high-level overview of the theoretical underpinning, we can dive deeper into the implementation of the three algorithms and experiment with their behavior. In the three following sub-sections, we will explain the steps of each algorithm. The complete steps and their respective pseudo-code can be found in the appendix below.

3.1 ε-Greedy

First, we implemented the ε-Greedy Algorithm. The ε-Greedy algorithm balances exploration and exploitation by mostly choosing the arm with the highest estimated reward. It occasionally picks a random arm with probability ε to explore other possible options. The algorithm starts by initializing the number of times each arm has been pulled, N_i and their estimated values, Q_i, to zero. At each time step, it then either selects a random arm or chooses the arm with the highest current estimate (depending on a comparison with the parameter ϵ). After selecting an arm and observing the reward r_t, the algorithm updates the count N_{a_t} and incrementally updates the estimated value Q_{a_t} using the formula

$$Q_{a_t} \leftarrow Q_{a_t} + \frac{r_t - Q_{a_t}}{N_{a_t}}.$$

This process is repeated for all T time steps and allows the algorithm to refine the value estimates of each arm based on the observed rewards.

3.2 UCB1

In a second step, we implemented the UCB1 Algorithm. The UCB1 algorithm takes uncertainty into account by favoring arms that either have high estimated rewards or have been chosen less often. This promotes the systematic exploration of less certain arms. Here as well, the algorithm begins by initializing the count N_i and estimated value Q_i for each arm to zero. At each time step, it first checks if there are any arms that have not been selected yet. If this is the case, it chooses one of them. Otherwise, it selects the arm a_t that maximizes the sum of its current estimate Q_i and a confidence term $\sqrt{\frac{2 \ln t}{N_i}}$,

which depends on the number of times the arm has been pulled and the current time step t. After observing the reward r_t, the algorithm updates the count N_{a_t} and the value estimate Q_{a_t} incrementally using the same formula as in the ε-Greedy Algorithm. This procedure is repeated for all T time steps, allowing the algorithm to maintain updated estimates while accounting for uncertainty in less frequently chosen arms.

3.3 Thompson Sampling

Finally, we implemented the Thompson Sampling Algorithm in a Bernouilli setting. Thompson Sampling uses a Bayesian approach and selects arms by sampling from the posterior distributions of their rewards (this balances exploration and exploitation through randomization). The algorithm starts by initializing the parameters α_i and β_i of a Beta distribution[1] for each arm to 1. At each time step, it samples a value θ_i from the Beta distribution of each arm and selects the arm a_t with the highest sampled value. After observing the reward r_t, it updates the corresponding Beta parameters: if $r_t = 1$, α_{a_t} is incremented by 1. Otherwise, β_{a_t} is incremented by 1. This process is repeated for all T time steps and allows the algorithm to maintain a posterior distribution for each arm and make decisions based on these updated beliefs. In other words, Thompson Sampling chooses arms in proportion to how likely they are to be optimal given the current evidence.

4. Technical Implementation of the Main Algorithms

4.1 Setting

To understand their functioning and behavior, we implemented and tested these three algorithms from scratch in Python. The experiments were carried out in a Bernoulli bandit setting, where each arm gives a reward of 1 with a fixed probability and 0 otherwise. We chose a three-armed bandit setup, meaning that the agent must repeatedly choose between three distinct actions, each associated with a different but unknown reward probability.

To test how well the algorithms perform under different conditions, we considered three levels of difficulty:

- Easy: rewards $[0.1, 0.5, 0.9]$ – large differences between arms, making the optimal arm easy to identify.
- Medium: rewards $[0.3, 0.5, 0.7]$ – moderate differences, requiring a balanced exploration.

[1]The beta distribution is used because it is the conjugate prior for the Bernoulli distribution.

- Hard: rewards $[0.48, 0.50, 0.52]$ – very small differences, where identifying the best arm is challenging.

For each algorithm and configuration, we ran 5,000 iterations per experiment, repeated each experiment 50 times to account for randomness and compared the performance of the three algorithms. For Epsilon-Greedy we used $\epsilon = 0.1$.

We measured three main metrics to evaluate performance: First, the **cumulative reward**. This gives us the total reward collected over time and provides an overall sense of how well the algorithm is performing. Then, the **optimal arm selection rate**, which is the fraction of times the algorithm picks the best arm. This shows how quickly and consistently the algorithm learns. Finally, we tracked the **cumulative regret**. This is the difference between always choosing the optimal arm and the actual reward of the algorithm and allows us to quantify the cost of exploration.

We also explored how the choice of epsilon affects the performance of the Epsilon-Greedy algorithm by testing $\epsilon \in \{0.01, 0.05, 0.1, 0.15, 0.2\}$.

The experiments' code can be accessed here:
`nicolasreichardt/stochastic-bandit-algorithms`.

In the repository, the three algorithms are implemented in a modular manner, with each algorithm contained in its own module within the `src` folder. The experiments described above are executed in the main notebook, `bandit_algorithms.ipynb`.

5. Results and Interpretation

These metrics were recorded for all three difficulty levels and visualized in Figure 1. Figure 1(a) corresponds to the easy setting, Figure 1(b) to the medium setting and Figure 1(c) to the hard setting.

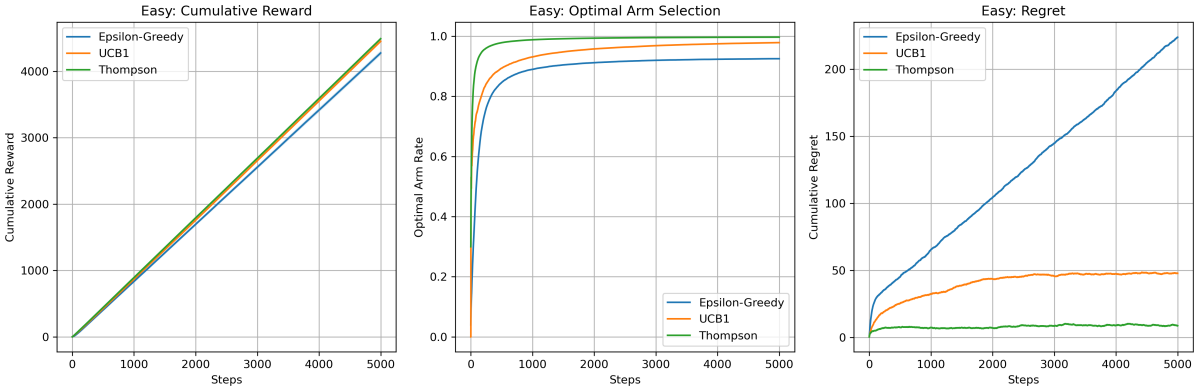
In the easy setting, all three algorithms perform similarly well in terms of cumulative reward, reaching around 4500 units by step 5000. However, differences become apparent when looking at how efficiently each algorithm balances exploration and exploitation. Thompson Sampling performs the best, reaching nearly 100% optimal arm selection by around step 1500, which shows that it identifies the optimal arm very quickly. UCB1 also performs well, reaching about 97% optimal arm selection by step 5000. Epsilon-Greedy converges more slowly and plateaus at around 93%. These differences are also reflected in the cumulative regret. Thompson Sampling accumulates the least regret (approximately 10 units), followed by UCB1 with around 50 units, while Epsilon-Greedy has the highest regret, exceeding 220 units. This indicates a more frequent selection of suboptimal arms.

In the medium setting, the performance differences between the algorithms become more noticeable. Thompson Sampling continues to outperform the others, maintaining

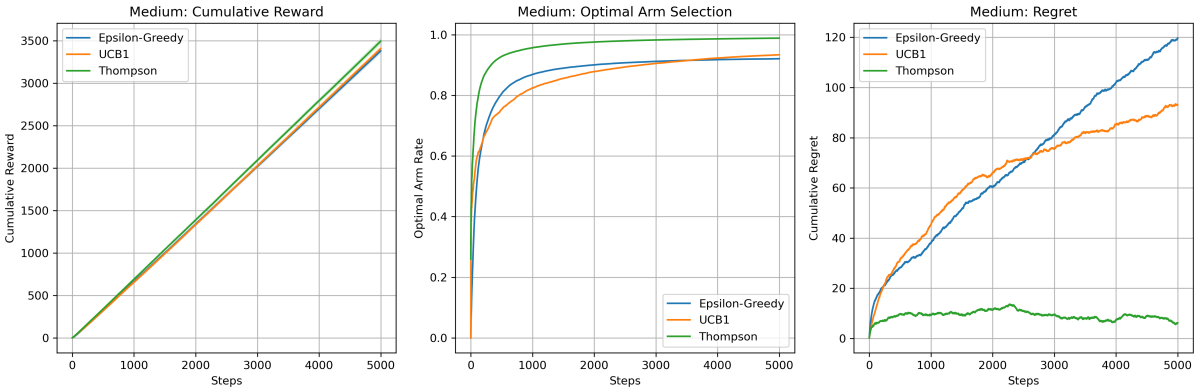
almost perfect optimal arm selection and very low cumulative regret (below 10 units). UCB1 still performs reasonably well, reaching about 93% optimal arm selection with cumulative regret around 92 units. Epsilon-Greedy struggles more in this setting, with an optimal arm selection leveling off at roughly 90% and cumulative regret increasing to around 118 units. However, all three algorithms achieve similar cumulative rewards (approximately 3400–3500 units). This suggests that moderate differences between arms still allow for reasonable reward accumulation even with less efficient exploration.

Finally, the hard setting highlights the largest performance gaps. Thompson Sampling again performs best and achieves the highest optimal arm selection rate (around 70% by step 5000) and the lowest cumulative regret (approximately 50 units). Epsilon-Greedy reaches about 60% optimal arm selection with slightly higher regret (around 58 units). UCB1 performs the worst in this scenario and plateaus at roughly 47% optimal arm selection and accumulating the highest regret (around 76 units). Interestingly, despite these differences, all three algorithms achieve nearly the same cumulative reward (around 2550 units). This means that when the differences between arms are small, suboptimal arm choices have less impact on total reward, even though regret still highlights differences in learning efficiency.

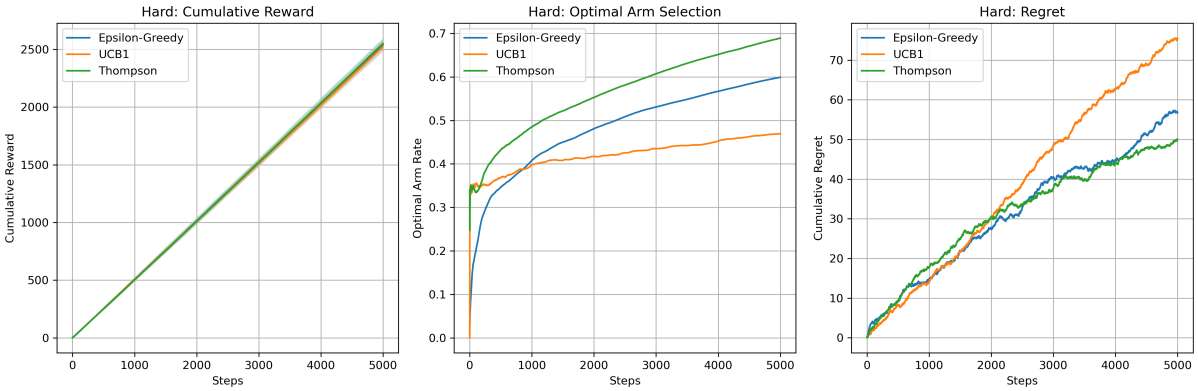
Overall, Thompson Sampling shows the most consistent and robust performance across all difficulty levels. It is particularly effective in easy and medium settings where it quickly identifies and exploits the optimal arm while keeping regret low. UCB1 performs competitively well in easier scenarios but struggles when the problem becomes more difficult and arm differences are more subtle. Epsilon-Greedy is more sensitive to problem difficulty and generally accumulates higher regret. These results suggest that Thompson Sampling provides a better balance between exploration and exploitation (especially when a clear optimal arm exists).



(a) Easy reward configuration



(b) Medium reward configuration



(c) Hard reward configuration

Figure 1: Results for different reward configurations

We also analyzed the effect of different ε values on the performance of the Epsilon-Greedy algorithm, as shown in Figure 2. A low value ($\varepsilon = 0.01$) leads to slow convergence, with the optimal arm selected only about 80% of the time by step 5000. Higher values ($\varepsilon = 0.1, 0.15, 0.2$) converge much faster, reaching 85–90% optimal arm selection within the first 1000–1500 steps due to increased exploration.

Despite faster early learning, higher ε values achieve slightly lower long-term cumulative rewards, since they continue to explore suboptimal arms even after the optimal one is identified. In contrast, $\varepsilon = 0.01$ attains marginally higher final rewards by reducing

unnecessary exploration.

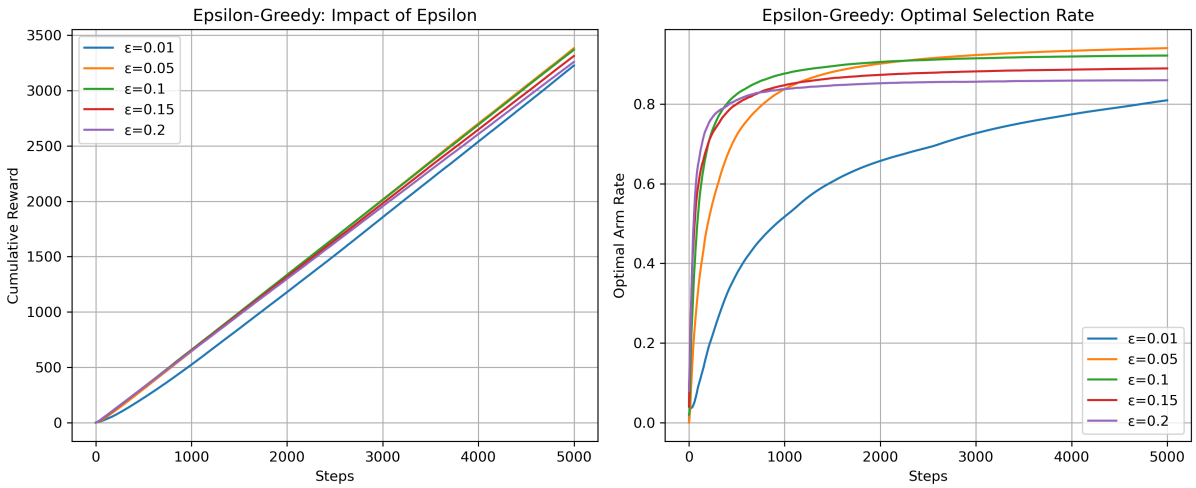


Figure 2: Epsilon sensitivity analysis.

6. Conclusion

This term paper presented a high-level introduction to stochastic multi-armed bandits by combining theory with a practical implementation and empirical evaluation of three classical algorithms: ε-Greedy, UCB1 and Thompson Sampling. By implementing each method from scratch, we illustrated how different approaches to the exploration–exploitation trade-off lead to different learning behaviors and performance characteristics.

The experimental results show that while all three algorithms achieve similar cumulative rewards, they differ notably in learning efficiency and regret. Thompson Sampling consistently performs best across difficulty levels, UCB1 works well when reward differences are clear and ε-Greedy provides a simple but less efficient baseline that is sensitive to parameter tuning.

References

Botosan, I.-A. and Bilokon, P. (2024). Optimal resource allocation using multi-armed bandits.

Clement, B., Roy, D., Oudeyer, P.-Y., and Lopes, M. (2015). Multi-armed bandits for intelligent tutoring systems.

Gigli, M. and Stella, F. (2025). Multi-armed bandits for performance marketing. *International Journal of Data Science and Analytics*, 20(1):151–165.

Li, L., Chu, W., Langford, J., and Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670.

Manickam, I., Lan, A. S., and Baraniuk, R. G. (2017). Contextual multi-armed bandit algorithms for personalized learning action selection. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6344–6348. IEEE.

Mehrotra, R., Xue, N., and Lalmas, M. (2020). Bandit based optimization of multiple objectives on a music streaming platform. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3224–3233.

Rafferty, A., Ying, H., and Williams, J. (2019). Statistical consequences of using multi-armed bandits to conduct adaptive educational experiments. *Journal of Educational Data Mining*, 11(1):47–79.

Schwartz, E. M., Bradlow, E. T., and Fader, P. S. (2017). Customer acquisition via display advertising using multi-armed bandit experiments. *Marketing Science*, 36(4):500–522.

Appendix

Pseudo-code for Epsilon-Greedy Algorithm.

Algorithm 1 Epsilon-Greedy Algorithm

Input: Number of arms K, exploration parameter ϵ
Initialize counts $N_i = 0$ and value estimates $Q_i = 0$ for all arms i
for $t = 1$ to T **do**
  if Uniform$(0, 1) < \epsilon$ **then**
    Select a random arm a_t
  else
    Select arm $a_t = \arg\max_i Q_i$
  end if
  Observe reward r_t
  $N_{a_t} \leftarrow N_{a_t} + 1$
  $Q_{a_t} \leftarrow Q_{a_t} + \frac{r_t - Q_{a_t}}{N_{a_t}}$
end for

Pseudo-code for UCB1 Algorithm.

Algorithm 2 UCB1 Algorithm

Input: Number of arms K
Initialize counts $N_i = 0$ and value estimates $Q_i = 0$ for all arms i
for $t = 1$ to T **do**
  if There exists an arm i with $N_i = 0$ **then**
    Select such an arm a_t
  else
    Select arm

$$a_t = \arg\max_i \left(Q_i + \sqrt{\frac{2 \ln t}{N_i}} \right)$$

  end if
  Observe reward r_t
  $N_{a_t} \leftarrow N_{a_t} + 1$
  $Q_{a_t} \leftarrow Q_{a_t} + \frac{r_t - Q_{a_t}}{N_{a_t}}$
end for

Pseudo-code for Thompson Sampling.

Algorithm 3 Thompson Sampling for Bernoulli Bandits

Input: Number of arms K
Initialize $\alpha_i = 1$, $\beta_i = 1$ for all arms i
for $t = 1$ to T **do**
  Sample $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$ for all arms
  Select arm $a_t = \arg\max_i \theta_i$
  Observe reward r_t
  if $r_t = 1$ **then**
    $\alpha_{a_t} \leftarrow \alpha_{a_t} + 1$
  else
    $\beta_{a_t} \leftarrow \beta_{a_t} + 1$
  end if
end for

AI use statement: Generative AI tools like GitHub Copilot and Claude were consulted for writing support, LaTeX formatting and general coding/linting support. All final answers were developed and verified by the author.